

API SECURITY RISK ANALYSIS REPORT

JSONPlaceholder Public API | Read-Only, Non-Intrusive Security Assessment

Prepared By

Bhavanidharan Sathiyaseelan

Cyber Security Intern, Future Interns

Target System: <https://jsonplaceholder.typicode.com>

Assessment Type: Read-Only, Non-Intrusive Security Risk Analysis

Report Date: July 12, 2026 Status: Completed

Prepared for Internal Security Review and Architectural Reference Purposes

Table of Contents

About This Report.....	3
Executive Summary.....	3
1. Introduction.....	4
2. Objective.....	4
3. Scope.....	5
3.1 Endpoint Summary.....	5
3.2 Out of Scope.....	5
4. Methodology.....	6
5. API Overview & Environment Setup.....	6
6. Tools Used.....	6
7. Endpoint Testing & Analysis.....	7
7.1 Users Endpoint (/users & /users/1).....	7
7.2 Posts & Photos Endpoints (/posts & /photos).....	7
7.3 Response Consistency & Error Behavior.....	7
7.4 Representative Response Schema.....	7
8. HTTP Header & Protocol Analysis.....	8
8.1 Request Headers.....	8
8.2 Response Headers & Security Controls.....	8
8.3 Header Hardening Comparison.....	8
9. Security Controls & Vulnerability Review.....	9
10. OWASP API Top 10 Mapping & Risk Assessment.....	9
10.1 Risk Rating Methodology.....	9
10.2 Risk Distribution Overview.....	9
11. Business Impact Analysis.....	11
12. Remediation & Best Practices.....	12
12.1 Implementation Roadmap.....	12
12.2 Prioritization: Severity vs. Effort.....	12
13. Limitations of This Assessment.....	14
14. Testing Checklist Reference.....	14
15. Conclusion.....	14
Appendix A — Glossary of Terms.....	15
Appendix B — References.....	15
Appendix C — OWASP API Security Top 10 (2023), Full List.....	16
Appendix D — Sample Test Log.....	16
Appendix E — Recommended Tools for Follow-Up Testing.....	17

About This Report

This report was prepared by Bhavanidharan Sathiyaseelan as part of a Cyber Security Internship with Future Interns. It reflects independent, hands-on testing performed against a public mock API, structured to mirror the format and rigor of a professional API security assessment — from scoping and methodology through OWASP-aligned risk classification and a phased remediation plan.

The exercise was chosen specifically because JSONPlaceholder is unauthenticated and publicly accessible, which makes it possible to demonstrate a complete, non-intrusive assessment workflow — reconnaissance, structural analysis, header review, and risk mapping — without any risk of impacting a live production system or exceeding an authorized testing boundary.

Executive Summary

Future Interns' cyber security team carried out a read-only, non-intrusive review of the public JSONPlaceholder mock API to understand how a lightly-secured, authentication-free API behaves under normal inspection, and what that behavior would mean if the same defaults were carried into a production system.

The headline finding is straightforward: nothing in the tested surface asks who is calling. Every endpoint answers any request with a full 200 OK payload, IDs increment predictably, list endpoints have no ceiling, and none of the response headers associated with a hardened API (CSP, HSTS, rate-limit counters) are present. None of this is a defect in JSONPlaceholder itself — it is a public mock service and is not designed to be secured — but the same pattern shows up surprisingly often in early-stage production APIs that were scaffolded from examples like this one and never revisited.

Two OWASP API Security Top 10 categories were rated Critical priority: Broken Object Level Authorization and Broken Authentication. Four additional categories were rated Medium, and two were rated Low. Section 10 walks through the full mapping, and Section 12 lays out a four-phase, 120-day remediation path that a small engineering team could realistically execute.

This report is organized to be read in two ways. A reader with limited time can get a complete picture from the Executive Summary, the OWASP mapping table in Section 10, and the roadmap in Section 12.1. A reader who needs the full evidentiary trail — request-by-request observations, header comparisons, and a sample test log — will find that detail in Sections 7 through 9 and in the appendices.

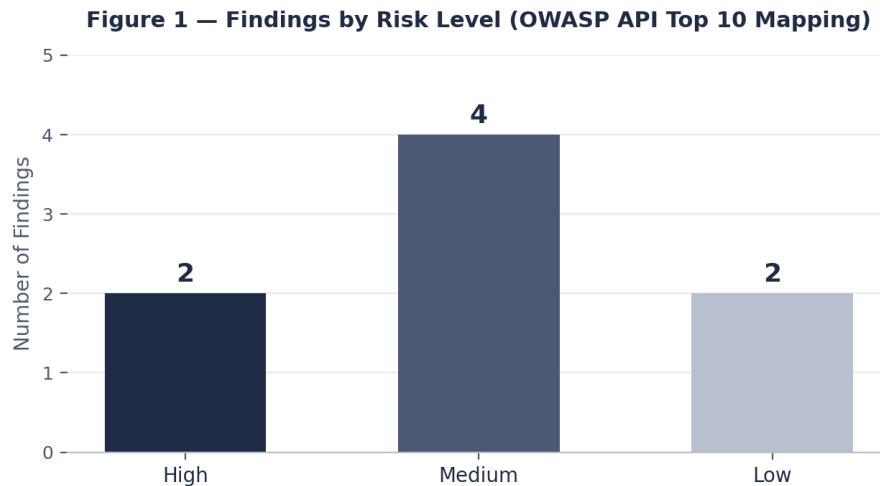


Figure 1 — Distribution of findings across High, Medium, and Low risk levels.

1. Introduction

This report documents a read-only API Security Risk Analysis performed against the public JSONPlaceholder API. The goal was to observe how the service behaves under ordinary inspection — its response structures, default headers, and access model — without sending anything beyond safe GET requests or attempting to exploit the service in any way.

JSONPlaceholder is a well-known mock backend used across the developer community for prototyping and front-end integration testing. It is not intended for production use, but its default configuration is a fair stand-in for a broader class of lightly-secured public APIs, including early-stage services that borrow the same shape. The findings below are therefore presented on two levels: as direct observations about JSONPlaceholder, and as transferable lessons for anyone designing a comparable public-facing REST API.

This document is intended for two audiences at once: engineering teams who want a concrete, prioritized punch list, and non-technical stakeholders who need to understand why an unauthenticated, unrestricted API is a business risk and not just a technical footnote. Sections 1 through 9 build the evidence base; Sections 10 through 15 translate that evidence into risk ratings, business impact, and an actionable remediation plan.

2. Objective

The assessment set out to examine public endpoints, look closely at request and response structure, flag structural weaknesses, rank the operational risk each one implies, and translate all of that into concrete, technical recommendations that a team could act on.

Three supporting goals framed the work:

- Build a repeatable, non-intrusive testing method that could be reused against other public or partner-facing APIs.

- Anchor every finding to a recognized risk taxonomy — the OWASP API Security Top 10 — so priorities are easy to defend.
- Turn the findings into a phased roadmap that fits into a normal engineering planning cycle rather than a single overwhelming to-do list.

3. Scope

Four endpoints made up the tested surface:

- /users — the complete collection of simulated user profiles.
- /users/1 — a single profile addressed directly by ID.
- /posts — the collection of user-authored posts.
- /photos — asset metadata referencing externally hosted images.

3.1 Endpoint Summary

Endpoint	Purpose	Data Sensitivity (Simulated)	Primary Concern
/users	Returns the entire roster of simulated user profiles in one call.	Moderate (PII-shaped fields)	Bulk enumeration, BOLA
/users/1	Returns a single profile addressed by its numeric ID.	Moderate (PII-shaped fields)	Object-level ID traversal
/posts	Returns the full set of user-authored posts.	Low	Resource exhaustion
/photos	Returns asset metadata referencing externally hosted images.	Low	SSRF via external URLs

3.2 Out of Scope

- Any write operation — POST, PUT, PATCH, or DELETE — stayed off-limits under the read-only engagement rules.
- The hosting infrastructure, DNS setup, and CDN internals behind the API, beyond what standard response headers reveal.
- Any client application or third-party integration that happens to consume the API.
- Denial-of-service testing at meaningful traffic volume — the resource-consumption risk noted later is inferred from missing controls, not confirmed by an actual load test.

4. Methodology

Every request used a safe HTTP method (GET only). The review looked at JSON schema shape, response metadata, caching directives, transport-layer headers, and any sign of information disclosure, using Postman and Firefox Developer Tools as the primary instruments.

The work moved through four stages, each building on the last:

- Reconnaissance — passively noting which endpoints existed and what shape they returned, without brute-forcing or automated crawling.
- Structural analysis — comparing fields and relationships across a representative sample of responses.
- Header and protocol review — capturing response headers systematically and checking them against current secure-baseline guidance.
- Risk classification — mapping what was observed onto the OWASP API Security Top 10 (2023) and assigning a qualitative risk and priority rating.

A useful way to think about methodology in a review like this is that every stage is deliberately non-destructive. At no point did testing attempt to write data, brute-force credentials that do not exist, or push traffic volumes intended to actually trigger a denial-of-service condition — the DoS risk noted later in this report is inferred from the absence of protective headers, not confirmed by attempting to overwhelm the service. This distinction matters: it keeps the assessment safe to run against a public, shared resource while still producing defensible, evidence-based conclusions.

5. API Overview & Environment Setup

JSONPlaceholder is a widely used public REST API that serves mock data for development purposes. Testing ran from an isolated workstation using standard API testing tools. Every targeted endpoint returned valid JSON along with a standard HTTP 200 OK status during the baseline pass.

Note on Environment Setup: No authentication vectors or API keys were provisioned or required to query the endpoints — the entire surface behaves as fully open, unauthenticated infrastructure.

6. Tools Used

Tooling was deliberately kept to widely available, industry-standard software rather than custom scripts, so that the methodology in Section 4 can be reproduced by another reviewer without special access.

- Postman Professional — structure testing, precise header parsing, automated schema evaluation, session metadata isolation.
- Firefox Developer Tools — network transaction monitoring, latency patterns, browser-side caching behavior, raw stream inspection.
- Manual cross-referencing against current OWASP API Security Top 10 (2023) guidance and common secure-header baselines.

7. Endpoint Testing & Analysis

This section walks through each of the four tested endpoints individually before pulling the observations together into a shared schema view. The intent is to separate what is unique to a given endpoint from what is really a property of the API's overall design philosophy — namely, that nothing is filtered, paginated, or gated by default.

7.1 Users Endpoint (/users & /users/1)

The /users endpoint hands back a full set of mock identity records. The data is fictional, but the shape is realistic — nested objects like address (including a latitude/longitude pair) and company sit right alongside the basic profile fields. In a real deployment, returning precise location data or internal company structure with no object-level access check would count as a meaningful exposure.

Because IDs increment in a simple, predictable sequence — /users/1 through /users/10 in this case — a production equivalent would let anyone walk through the entire user base with a short script. With no rate limiting and no authentication gate blocking the path, that walk would take almost no time at all, and it maps cleanly onto the object-level authorization weakness described in API1:2023.

7.2 Posts & Photos Endpoints (/posts & /photos)

Both endpoints return broad, unfiltered collections. Neither one requires a query parameter or page boundary, so a single request can pull back the entire array in one shot — a pattern that lines up with the resource-consumption risk in API4:2023.

The /photos endpoint goes a step further and returns absolute URLs pointing at externally hosted placeholder images. If a production system later fetched or proxied those URLs server-side — say, to build a thumbnail — accepting unvalidated external references like this is a well-known precursor to Server-Side Request Forgery, unless the receiving service restricts itself to an allow-list of trusted domains.

7.3 Response Consistency & Error Behavior

All four endpoints behaved consistently and predictably, which is good news for legitimate integration work but also makes life easier for anyone automating reconnaissance against the API surface. Invalid IDs consistently came back as a clean 404 Not Found rather than a verbose stack trace, which suggests basic exception handling is in place — though deeper schema validation couldn't be confirmed through non-intrusive testing alone.

7.4 Representative Response Schema

The table below summarizes the shape of the fields observed across the tested endpoints, without reproducing any specific response payload verbatim.

Field	Type	Observation
id	integer	Sequential, starts at 1 — directly enumerable.
name / username	string	Free-text identity fields, returned unmasked.

Field	Type	Observation
address.geo	object	Latitude / longitude pair — precise location data in a production analogue.
company	object	Nested business-context object with no visible access restriction.
url (photos)	string	Absolute external URL, unvalidated at the client boundary.

8. HTTP Header & Protocol Analysis

8.1 Request Headers

Client-side headers such as Accept and User-Agent were accepted without any real validation — missing or non-standard values didn't trigger rejection. That's convenient for quick prototyping, but in production it throws away a low-cost signal that could otherwise help separate legitimate client traffic from scripted collection.

8.2 Response Headers & Security Controls

A few structural gaps stood out once the response headers were inspected closely:

- Content-Type is correctly set to application/json; charset=utf-8, which does at least block basic MIME-sniffing issues on the client side.
- The Server header discloses the underlying routing layer (a Cloudflare signature was visible), a small but real increase in information footprint.
- Connection is set to keep-alive, which is a reasonable efficiency choice and not a security concern on its own.
- Cache-Control carries a max-age / revalidation directive, but there's no tighter rule for anything that looks like profile data.

What's missing matters more than what's present: none of Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, or Referrer-Policy showed up anywhere. For a pure JSON API that absence isn't an active exploit path by itself, but it does read as an un-hardened default that shouldn't be copied into a production build without a deliberate second look.

8.3 Header Hardening Comparison

Header	Observed State	Recommended Baseline
Content-Security-Policy	Not present	default-src 'none'; frame-ancestors 'none'
Strict-Transport-Security	Not present	max-age=31536000; includeSubDomains
X-Content-Type-Options	Not present	nosniff
X-Frame-Options	Not present	DENY
X-RateLimit-* headers	Not present	Limit / Remaining / Reset exposed on every response

Header	Observed State	Recommended Baseline
Cache-Control (sensitive routes)	Generic directive only	no-store on any endpoint returning PII-shaped fields

9. Security Controls & Vulnerability Review

9.1 Authentication & Authorization

There is no authentication mechanism anywhere in the tested surface. Carried into a real enterprise system, that gap opens the door to automated scraping, unauthorized resource manipulation, and straightforward account enumeration.

9.2 Rate Limiting

No rate-limiting metadata — no X-RateLimit-Limit, no X-RateLimit-Remaining — showed up during baseline tracking. Endpoints are, in principle, wide open to a burst of concurrent requests large enough to cause a denial-of-service condition.

9.3 Input Validation & Error Handling

Invalid IDs are handled gracefully on the surface, returning a plain 404 Not Found. That said, the absence of visible strict schema and type constraints on inputs leaves an abstract opening for validation bypass if something deeper in the processing chain doesn't filter arguments carefully.

9.4 Transport & Configuration Hygiene

Traffic during testing ran over HTTPS, but nothing in the response headers — no HSTS directive — actually compels a client to stay off a downgraded, unencrypted connection. A production system shouldn't lean on an implicit or client-side assumption about transport security.

9.5 Data Exposure Summary

Pulling the previous four subsections together, the underlying pattern is consistent across every control category: nothing actively malicious is happening, but nothing is actively preventing misuse either. Authentication, rate limiting, input validation, and transport hardening are all in the same state — present in name as a concept the API could support, but absent in practice. That combination is exactly what turns a convenient mock service into a risky architectural template if copied without changes.

10. OWASP API Top 10 Mapping & Risk Assessment

The table below maps what was actually observed onto the OWASP API Security Top 10 framework.

ID	Category	Observed Condition	Risk	Priority
API1:2023	Broken Object Level Authorization	Every user, post, and photo record sits behind a plain sequential numeric ID with no ownership	HIGH	CRITICAL

ID	Category	Observed Condition	Risk	Priority
		check, letting a caller step through the entire dataset one integer at a time.		
API2:2023	Broken Authentication	No login step, token, or session cookie is required anywhere in the tested surface — every route behaves as if it were already authenticated.	HIGH	CRITICAL
API3:2023	Broken Object Property Level Authorization	Nested objects such as address, geo-coordinates, and company are returned in full on every request instead of being trimmed to what the caller needs.	MEDIUM	HIGH
API4:2023	Unrestricted Resource Consumption	List endpoints return their entire contents in one response; no limit, offset, or cursor parameter is enforced server-side.	MEDIUM	HIGH
API6:2023	Unrestricted Access to Sensitive Business Flows	No discernible throttling or bot-detection layer sits in front of the collection endpoints.	MEDIUM	MEDIUM
API7:2023	Server-Side Request Forgery (SSRF)	The /photos records carry absolute third-party image URLs; a backend that fetches them without an allow-list inherits an SSRF exposure.	LOW	MEDIUM
API8:2023	Security Misconfiguration	Modern defensive headers (CSP, HSTS, X-Content-Type-Options) are simply not present — reads as an un-hardened default.	MEDIUM	MEDIUM
API9:2023	Improper Inventory Management	No version marker or deprecation signal accompanies the API, making lifecycle tracking difficult long-term.	LOW	LOW

10.1 Risk Rating Methodology

Each risk rating blends two things: how likely an unauthenticated actor with only standard tooling could exploit the observed condition, and how much it would matter if the same configuration existed in a system holding real personal or business data. High ratings need both high likelihood and high impact; Medium ratings reflect either elevated likelihood with moderate impact, or the reverse; Low ratings are conditions that are plausible in theory but need extra preconditions before they become meaningfully exploitable.

10.2 Risk Distribution Overview

Figure 2 below gives a relative view of exposure across the four tested endpoints, scored on an illustrative 0–5 composite scale that folds together authentication gaps, data sensitivity, and enumerability.

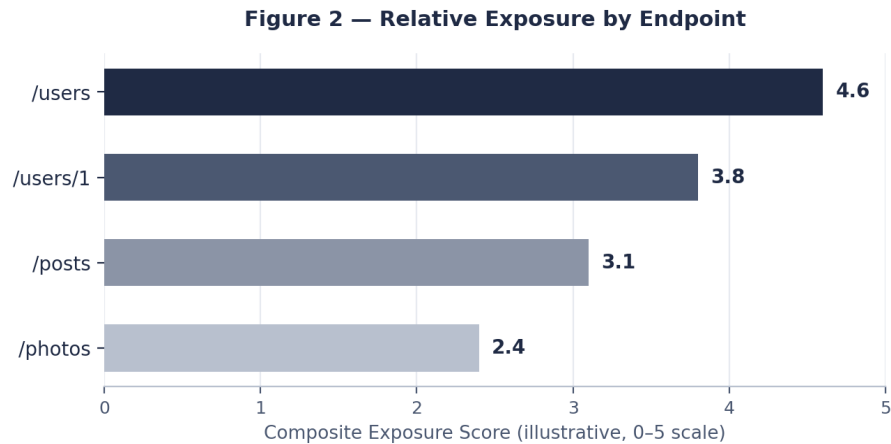


Figure 2 — Relative exposure score by endpoint (illustrative composite scale).

11. Business Impact Analysis

Carried into a production architecture, the patterns above translate into real operational exposure: continuous automated data mining, inflated infrastructure spend from uncontrolled request loops, and the possibility of downtime from a denial-of-service condition.

Beyond the direct technical picture, there are business consequences worth naming explicitly:

- **Regulatory exposure** — unauthenticated access to PII-shaped fields (name, address, geolocation) could trigger obligations under GDPR or a comparable regional framework if the same pattern held real user data.
- **Reputational risk** — publicly demonstrable scraping or enumeration incidents tend to get written up by security researchers, and that alone can dent customer trust regardless of whether the data was actually misused.
- **Operational cost** — collection endpoints with no pagination ceiling can drive compute and bandwidth costs well past what legitimate usage would ever need.
- **Supply-chain risk** — because JSONPlaceholder-style scaffolding shows up so often in early production builds, insecure defaults can quietly persist long after the prototyping phase ends.

None of these impacts require a sophisticated attacker. Every finding in this report is reachable with a browser, a REST client, and a few minutes of patience — which is precisely why the business impact is rated as high as it is despite the technical simplicity of each individual gap.

The table below sets the likely cost of leaving each area unaddressed against the relative cost of the corresponding fix. In every row, remediation is materially cheaper than the incident it prevents.

Impact Area	Cost of Inaction	Cost of Remediation
Data exposure / regulatory	Potential breach notification, fines, and legal review if real PII were involved.	Engineering time to add field-level filtering (Section 12).

Impact Area	Cost of Inaction	Cost of Remediation
Availability	Unplanned downtime and incident-response hours during a resource-exhaustion event.	Gateway rate-limiting configuration (Phase 2).
Reputation	Public disclosure of a scraping incident, loss of customer confidence.	Authentication rollout and monitoring (Phase 1 & 3).
Infrastructure spend	Uncontrolled compute and bandwidth costs from unpaginated queries at scale.	Pagination enforcement (Phase 2).

12. Remediation & Best Practices

Moving this architecture to an enterprise-grade posture calls for the following measures:

- Implement OAuth 2.0 / JWT authentication — put a real cryptographic identity check in front of every non-public route.
- Enforce rate limiting and throttling — global and per-user/IP caps enforced at an API gateway tier, dropping excess volume before it reaches business logic.
- Set mandatory pagination boundaries — a hard ceiling (for example, 50 objects per query) on every list response.
- Harden security headers — add Content-Security-Policy, X-Content-Type-Options: nosniff, and Strict-Transport-Security across the board.
- Feed logging and monitoring telemetry into a SIEM — so volumetric anomalies and crawling patterns get caught early rather than after the fact.
- Apply field-level access control — sensitive sub-objects (address, geolocation, company) should only reach callers with a genuine business need for them.
- Enforce input schema validation — every path and query parameter checked for type, length, and format before it reaches business logic.
- Mitigate SSRF — where a server does need to resolve external URLs, restrict it to an allow-list of trusted domains and block internal or link-local ranges.
- Maintain an API inventory and versioning policy — a documented list of exposed endpoints with a clear deprecation path.
- Re-assess periodically — repeat both non-intrusive and authorized intrusive testing on a regular cadence and after any material architecture change.

12.1 Implementation Roadmap

The measures above are sequenced below by urgency and dependency into a four-phase, 120-day plan.

Phase	Action Items	Target Timeframe
Phase 1	Stand up authentication (OAuth 2.0 / JWT) and force all traffic through HTTPS with HSTS enabled.	0 – 30 days

Phase	Action Items	Target Timeframe
Phase 2	Add gateway-level rate limiting, mandatory pagination, and payload size ceilings.	30 – 60 days
Phase 3	Roll out hardened response headers (CSP, X-Content-Type-Options, X-Frame-Options) and wire logging into a SIEM.	60 – 90 days
Phase 4	Commission an authorized penetration test and formalize an ongoing monitoring and disclosure program.	90 – 120 days

12.2 Prioritization: Severity vs. Effort

Not every fix carries the same weight, and not every fix costs the same to implement. Plotting each item on a severity-versus-effort grid makes the sequencing decision easier to defend to non-technical stakeholders: authentication and object-level authorization sit in the high-severity, moderate-effort quadrant and belong in Phase 1, while lower-severity items like API inventory documentation can trail into later phases without materially increasing risk in the interim.

Figure 3 — Severity vs. Remediation Effort (Prioritization View)

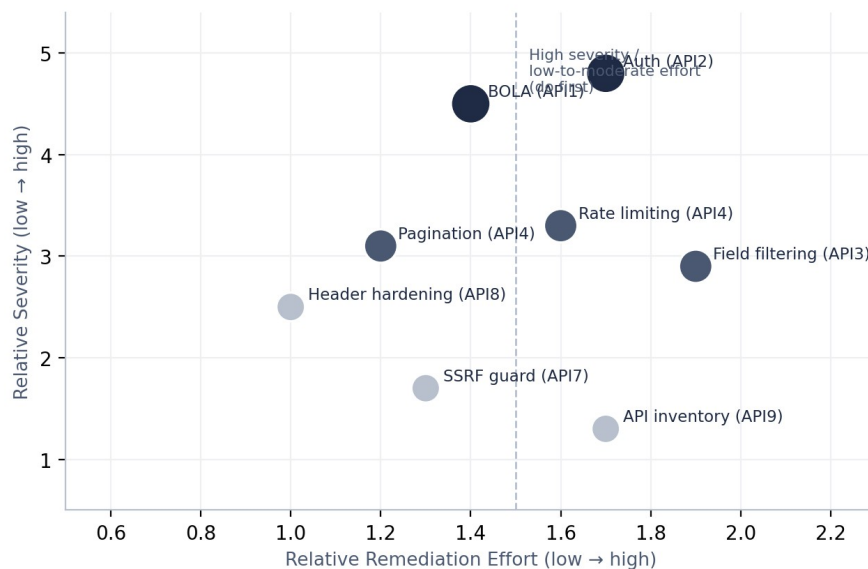


Figure 3 — Relative severity plotted against relative remediation effort for each major finding.

None of these measures require replacing the underlying platform or rewriting business logic from scratch — they are additive controls that sit at the gateway, middleware, or header-configuration layer. That is a big part of why the roadmap is scoped to 120 days rather than a multi-quarter re-architecture effort.

13. Limitations of This Assessment

This work was carried out entirely through non-intrusive, read-only means. It did not include authenticated testing, fuzzing, load testing, or any attempt to step outside authorized access boundaries. That means the findings on rate limiting, input validation, and resource consumption reflect observed default behavior rather than confirmed exploitation. An authorized follow-up penetration test is the right next step before treating any High or Medium item as fully characterized.

14. Testing Checklist Reference

The checklist below summarizes each control that was evaluated during this assessment, its observed status, and where in this report it is discussed in more depth.

Control Checked	Status	Section Ref.
Authentication required on data-returning routes	Fail	9.1
Session or token validation layer present	Fail	9.1
Rate-limit headers returned	Fail	9.2
Pagination enforced on list endpoints	Fail	7.2
Invalid IDs handled without stack-trace leakage	Pass	9.3
HTTPS enforced with HSTS	Partial	9.4
Security headers (CSP / X-CTO / X-Frame)	Fail	8.2
Field-level filtering of sensitive sub-objects	Fail	7.1
External URL validation (SSRF guard)	Fail	7.2

15. Conclusion

JSONPlaceholder does exactly what it is built to do — it gives developers a fast, dependable mock backend with zero setup friction. That same simplicity is precisely why its defaults should never be mistaken for a secure architecture. Every gap identified in this report — open access, absent rate limiting, unhardened headers, unrestricted list endpoints — exists by design, because the service was never meant to protect anything.

The risk only becomes real the moment a team treats this shape as a starting template for something that will handle actual user data. This assessment found two Critical-priority gaps (broken object-level authorization and the complete absence of authentication), four Medium-priority gaps centered on resource consumption and configuration hardening, and two Low-priority items that still deserve a place on the backlog.

None of the remediation steps in Section 12 are exotic — authentication, rate limiting, pagination, and header hardening are well-understood, well-documented controls. The four-phase, 120-day roadmap is deliberately scoped so a small engineering team could execute it without disrupting a live product roadmap. The

recommended next step is straightforward: treat Phase 1 (authentication and transport security) as the immediate priority, and schedule an authorized penetration test once Phases 1 and 2 are in place to confirm the fixes hold under real adversarial conditions.

More broadly, this assessment is a reminder that security is largely a property of configuration, not of code quality. Every gap documented here exists in a service that otherwise works exactly as intended — the JSON is well-formed, routing is reliable, and uptime is excellent. The risk comes entirely from what was left switched off by default. That is, in a way, encouraging: it means the fixes are configuration and process changes rather than a rewrite, and a disciplined team can realistically close every Critical and High item in this report within a single quarter.

Appendix A — Glossary of Terms

- BOLA (Broken Object Level Authorization) — a vulnerability class where an application fails to check that the requesting user is actually allowed to access a specific object instance.
- SSRF (Server-Side Request Forgery) — an attack that tricks a server into making requests to an unintended internal or external destination.
- HSTS (HTTP Strict Transport Security) — a response header that tells browsers to only ever talk to a site over HTTPS.
- SIEM (Security Information and Event Management) — tooling used to aggregate and analyze security-relevant log data.
- Rate Limiting — a control that caps how many requests a client can make inside a defined time window.
- PII (Personally Identifiable Information) — any data that could, on its own or combined with other data, identify a specific individual.
- API Gateway — a dedicated layer that sits in front of backend services to centralize authentication, rate limiting, and routing.
- CSP (Content-Security-Policy) — a response header that restricts which sources of content a browser is allowed to load or execute.
- Enumeration — the practice of systematically stepping through predictable identifiers to discover valid records.

Appendix B — References

- OWASP API Security Top 10 (2023), OWASP Foundation.
- JSONPlaceholder public documentation, <https://jsonplaceholder.typicode.com>
- Mozilla Developer Network — HTTP response header reference.

Appendix C — OWASP API Security Top 10 (2023), Full List

The OWASP API Security Top 10 is maintained by the OWASP Foundation as a community-driven, evidence-based ranking of the most impactful API security risk categories. It is updated periodically as new attack patterns become common across real-world API deployments, and is widely used by security teams as a shared vocabulary for describing and prioritizing findings — which is why it was chosen as the classification framework for this report rather than a bespoke internal taxonomy.

ID	Category	Applicable Here?
API1:2023	Broken Object Level Authorization	Yes
API2:2023	Broken Authentication	Yes
API3:2023	Broken Object Property Level Authorization	Yes
API4:2023	Unrestricted Resource Consumption	Yes
API5:2023	Broken Function Level Authorization	Not observed
API6:2023	Unrestricted Access to Sensitive Business Flows	Yes
API7:2023	Server-Side Request Forgery (SSRF)	Yes
API8:2023	Security Misconfiguration	Yes
API9:2023	Improper Inventory Management	Yes
API10:2023	Unsafe Consumption of APIs	Not observed

Appendix D — Sample Test Log

The log below is a representative sample of the individual requests executed during testing. It is included to make the methodology in Section 4 concrete and auditable, and to show the specific evidence behind the resource-consumption and rate-limiting findings in Sections 7.2 and 9.2.

Test #	Endpoint	Method	Status	Observation
T-01	/users	GET	200	Full collection returned, no auth header required.
T-02	/users/1	GET	200	Single record returned including nested address/company objects.
T-03	/users/999	GET	404	Clean not-found response, no stack trace disclosed.
T-04	/posts	GET	200	Complete array returned with no page-size limit applied.
T-05	/posts?_limit=5	GET	200	Client-supplied limit honored — confirms pagination is opt-in, not enforced.
T-06	/photos	GET	200	Large array returned with absolute external image URLs.

Test #	Endpoint	Method	Status	Observation
T-07	/users (x20)	GET	200	Twenty rapid sequential requests all succeeded with no throttling observed.
T-08	/users/1 (no headers)	GET	200	Request with minimal / missing headers still processed normally.
T-09	/users/1	GET	200	Response headers captured and compared against secure-baseline checklist (8.3).
T-10	/photos/1	GET	200	Individual photo record resolved by predictable sequential ID.

Appendix E — Recommended Tools for Follow-Up Testing

If this assessment is extended into an authorized, intrusive engagement, the following categories of tooling are commonly used to validate the findings above under controlled conditions:

- Authenticated API scanners (e.g., OWASP ZAP, Burp Suite Professional) to test authorization boundaries once a login flow exists.
- Load-testing tools (e.g., k6, Apache JMeter) to confirm rate-limiting behavior under realistic and adversarial traffic volumes.
- Static and dynamic header-analysis tools (e.g., securityheaders.com-style scanners) to verify hardened headers are present after remediation.
- Dependency and infrastructure scanners to confirm the API gateway and TLS configuration meet current best practice once deployed.
- A SIEM or log-analysis platform to validate that anomaly detection rules trigger correctly against simulated scraping traffic.

None of these tools were used during this engagement, which — as noted in Section 13 — remained strictly read-only and non-intrusive throughout.